



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE COMPUTO

UNIDAD DE APRENDIZAJE: COMPUTACIÓN GRAFICA

TAREA 4: ALGORITMOS PARA CALCULAR LAS
TRIANGULACIONES DE DELAUNAY

NOMBRE: FARRERA MENDEZ EMMANUEL SINAI

PROFESOR: PALMA OROZCO ROSAURA

GRUPO: 6CM3

FECHA DE ENTREGA: 05 DE ABRIL DEL 2024

Índice

Algoritmos para calcular las triangulaciones de Delaunay.	2
Iterative flipping from any triangulation.	2
Funcionamiento.	2
Ventajas	3
Desventajas	3
Complejidad	3
Plane sweep	4
Funcionamiento	4
Ventajas	5
Desventajas	5
Complejidad	6
Randomized incremental construction.	6
Funcionamiento	6
Ventajas	7
Desventajas	7
Complejidad	7
Conversion from Voronio Diagram.	7
Complejidad	9
Bibliografía	10

Algoritmos para calcular las triangulaciones de Delaunay.

Existen diversos algoritmos para calcular las triangulaciones de Delaunay. En este documento voy a explicar 4 de los algoritmos más utilizados que son:

- By iterative flipping from any triangulation
- By plane sweep
- By randomized incremental construcción.
- By conversion from Voronio Diagram.

Iterative flipping from any triangulation.

El volteado iterativo, también conocido como el algoritmo de volteado de aristas, es una técnica utilizada para transformar cualquier triangulación inicial de un conjunto de puntos en una triangulación de Delaunay.

Funcionamiento.

1. Arista volteable:

- El algoritmo comienza con cualquier triangulación inicial del conjunto de puntos.
- Una arista en la triangulación se considera "volteable" si satisface la siguiente condición:
 - Los dos triángulos adyacentes a la arista forman un cuadrilátero convexo cuando se combinan.

2. Volteado de arista:

- Si se encuentra una arista volteable, el algoritmo realiza un "volteado de arista".
- Esto implica eliminar la arista volteable y reemplazarla con la otra diagonal del cuadrilátero convexo formado por los dos triángulos adyacentes.
- En esencia, esto crea dos nuevos triángulos a partir de los cuatro originales.

3. Iteración y terminación:

- El algoritmo itera a través de la triangulación, verificando cada arista para ver si

cumple la condición de ser volteable.

- Si se encuentra una arista volteable, se voltea y el proceso se reinicia desde el principio.
- La iteración continúa hasta que no se encuentren más aristas volteables. En este punto, se garantiza que la triangulación resultante sea una triangulación de Delaunay.

Ventajas

- Este método es relativamente sencillo de implementar.
- Se puede aplicar a cualquier triangulación inicial, independientemente de su calidad.

Desventajas

- La complejidad temporal en el peor de los casos puede ser alta, especialmente para conjuntos de datos grandes.
- El orden en el que se revisan las aristas puede afectar la eficiencia del algoritmo.

Complejidad

La complejidad del algoritmo de volteado iterativo para calcular triangulaciones de Delaunay depende del método específico utilizado y de la distribución de los puntos de entrada. En el peor de los casos, la complejidad puede ser alta.

Caso desfavorable:

- Se ha demostrado que el algoritmo de volteado iterativo tiene una complejidad temporal en el peor de los casos de $O(n^4)$, donde n es el número de puntos de entrada.
- Esto puede ocurrir en escenarios específicos donde la secuencia de volteos realizados por el algoritmo no converge rápidamente hacia una triangulación de Delaunay.

Comportamiento promedio:

- En promedio, y para la mayoría de las distribuciones de puntos comunes, el algoritmo de volteado iterativo suele tener una complejidad temporal mucho mejor que el caso desfavorable.
- Se estima que, en promedio, la complejidad está alrededor de $O(n \log n)$.

Plane sweep

Los algoritmos de barrido de plano (sweepline) son otro enfoque eficiente para calcular triangulaciones de Delaunay.

Funcionamiento.

1. Preprocesamiento:

- Ordena todos los puntos del plano por su coordenada x (en orden creciente).

2. Línea de barrido:

- Imagina una línea vertical (línea de barrido) que comienza en el extremo izquierdo y se mueve continuamente hacia la derecha, cubriendo finalmente todo el plano.

3. Cola de eventos:

- Mantén una estructura de datos (como una cola de prioridad) llamada cola de eventos. Esta cola contiene los próximos eventos:
 - Eventos de puntos: Representan puntos encontrados por la línea de barrido, ordenados por su coordenada x.
 - Eventos de círculo: Representan circuncírculos de triángulos que podrían voltearse durante el barrido.

4. Proceso de barrido:

- Mientras la línea de barrido no haya llegado al final:
 - Saca el evento del frente de la cola.
 - Evento de punto:
 - Si es un evento de punto, localiza su posición en la triangulación actual (mantenida debajo de la línea de barrido).
 - Crea nuevos triángulos conectando el punto con sus dos vecinos más cercanos identificados a partir de la estructura de datos debajo de la línea de barrido.
 - Agrega eventos de círculo para los triángulos recién formados a la cola (estos círculos podrían violar la condición de Delaunay más adelante).

- Evento de círculo:
 - Si es un evento de círculo, verifica si el círculo realmente viola la condición de Delaunay (es decir, contiene otro punto dentro).
 - Si ocurre una violación, realiza un "volteo de arista" para rectificar la triangulación y actualiza la estructura de datos en consecuencia.
 - Elimina cualquier otro evento de círculo asociado con los triángulos volteados, ya que se vuelven irrelevantes.

5. Triangulación final:

- Después de procesar todos los eventos, la estructura de datos debajo de la línea de barrido representa la triangulación final de Delaunay.

Ventajas

- Predictibilidad: La complejidad temporal predecible lo hace ideal para conjuntos de datos grandes donde el rendimiento es crucial.
- Robustez: Funciona bien con cualquier distribución de puntos, sin sufrir los mismos problemas de eficiencia que el volteado iterativo en algunos casos específicos.
- Facilidad de implementación: La lógica del algoritmo es relativamente sencilla, lo que facilita su implementación en comparación con otros métodos más complejos.

Desventajas

- Memoria: El algoritmo de barrido de plano requiere más memoria que el volteado iterativo, ya que necesita mantener la estructura de datos de la triangulación y la cola de eventos durante todo el proceso.
- Constante de complejidad: Aunque la complejidad temporal del barrido de plano es asintóticamente mejor que la del volteado iterativo, la constante de complejidad puede ser mayor en la práctica, lo que significa que puede ser más lento para conjuntos de datos pequeños.
- Implementación optimizada: La eficiencia del barrido de plano depende de la implementación de la estructura de datos y las técnicas de barrido utilizadas, lo que requiere un mayor esfuerzo de desarrollo para lograr un rendimiento óptimo.

Complejidad

El algoritmo de barrido de plano para la triangulación de Delaunay tiene una complejidad temporal de $O(n \log n)$, donde n es el número de puntos. En general, se considera más eficiente que el peor de los casos del volteado iterativo ($O(n^4)$).

Randomized incremental construction.

La construcción incremental aleatorizada (RIC, por sus siglas en inglés) es un método poderoso para construir triangulaciones de Delaunay.

Funcionamiento

1. Inicialización:

- Comienza con una triangulación vacía, usualmente un triángulo que abarca todos los puntos.

2. Inserción aleatorizada:

- Inserta los puntos uno por uno en orden aleatorio. Este es el aspecto clave de la aleatorización.

3. Comprobación de la propiedad de Delaunay local:

- Después de insertar un nuevo punto, verifica si viola la condición de Delaunay con algún triángulo en la triangulación actual.
 - La condición de Delaunay establece que ningún punto debe estar dentro de la circunferencia de cualquier otro triángulo.

4. Volteo de triángulos (si es necesario):

- Si se encuentra una violación, realiza una serie de "volteos" en triángulos adyacentes para restaurar la condición de Delaunay.
 - Un volteo implica reemplazar una diagonal de un triángulo no-Delaunay con la otra diagonal, creando efectivamente dos nuevos triángulos.

5. Iteración:

- Repite los pasos 2-4 para cada punto hasta que se inserten todos los puntos.

Ventajas

- Simplicidad: El concepto central es relativamente sencillo de entender e implementar.
- Adaptabilidad: RIC funciona bien para diversas distribuciones de puntos, incluidas aquellas que podrían causar problemas a los algoritmos deterministas.

Desventajas

- Aleatoriedad: La dependencia del orden aleatorio introduce un ligero elemento de incertidumbre. Si bien la complejidad esperada es buena, existe una pequeña posibilidad de encontrar un escenario en el peor de los casos con una complejidad mayor.
- Estructuras de datos: La implementación eficiente requiere estructuras de datos adecuadas para administrar la triangulación y localizar de manera eficiente los triángulos afectados durante los volteos.

Complejidad

La complejidad esperada de $O(n \log n)$ en 2D se basa en la aleatoriedad de la inserción de puntos.

El peor caso de $O(n^2)$ en 2D puede ocurrir en casos específicos con configuraciones degeneradas de puntos.

En dimensiones superiores, la complejidad aumenta significativamente.

Las estructuras de datos eficientes pueden mejorar la complejidad práctica del algoritmo.

Conversion from Voronio Diagram.

Este algoritmo es un proceso complejo ya que tenemos dos aspectos importantes que tener en cuenta.

- Dualidad: Un diagrama de Voronoi y su triangulación de Delaunay correspondiente se consideran estructuras duales. Esto significa que contienen información relacionada, pero representan aspectos diferentes del conjunto de puntos.
- Pérdida de información: Un diagrama de Voronoi se centra en las áreas de influencia

alrededor de cada punto. No codifica explícitamente la información de conectividad requerida para una triangulación.

Sin embargo, existen técnicas para extraer la triangulación de Delaunay a partir de un diagrama de Voronoi.

1. Localización de puntos:

- Recorre cada punto en el diagrama de Voronoi.
- Para cada punto, utiliza algoritmos de localización de puntos (por ejemplo, punto-en-polígono) para determinar a qué celda de Voronoi pertenece.
- Los bordes entre celdas de Voronoi vecinas (aquellas que comparten un límite) corresponden a los bordes en la triangulación de Delaunay.
- Al identificar todos estos bordes de conexión, puedes reconstruir la triangulación de Delaunay.

2. Algoritmos de transformación por dualidad:

- Existen algoritmos especializados que transforman directamente un diagrama de Voronoi en su triangulación de Delaunay. Estos algoritmos aprovechan la relación matemática entre las dos estructuras.
- Algunos ejemplos incluyen el algoritmo de barrido de Lee-Preparata o el algoritmo de barrido de Fortune (que puede funcionar en ambas direcciones, convirtiendo de Voronoi a Delaunay o viceversa).

Elegir el método correcto:

- Si ya tienes un diagrama de Voronoi y necesitas extraer la triangulación de Delaunay por razones específicas, entonces los algoritmos de localización de puntos o transformación por dualidad podrían ser necesarios.
- Si solo necesitas la triangulación de Delaunay, considera calcularla directamente utilizando algoritmos dedicados como los mencionados anteriormente. Esto puede ser más eficiente, especialmente para conjuntos de datos grandes.

Complejidad.

1. Localización de puntos:

- Complejidad: $O(n * m)$, donde n es el número de puntos en el diagrama de Voronoi y m es la complejidad promedio de localizar un punto dentro de una celda de Voronoi (generalmente logarítmica en el número de lados de la celda).
- Explicación:
 - Se debe iterar sobre cada punto del diagrama (n veces).
 - Para cada punto, la localización dentro de una celda de Voronoi puede requerir un algoritmo como "punto-en-polígono" cuya complejidad varía según la forma de la celda (promedio m).

2. Algoritmos de transformación por dualidad:

- Complejidad: $O(n \log n)$, donde n es el número de puntos en el diagrama de Voronoi.
- Explicación:
 - Estos algoritmos aprovechan la relación matemática entre las estructuras y están diseñados para ser eficientes en el tamaño del conjunto de puntos.
 - Ejemplos como el algoritmo de barrido de Lee-Preparata o el de Fortune tienen una complejidad logarítmica promedio.

Bibliografia

[Tutorial] Voronoi Diagram and Delaunay Triangulation in $O(n \log n)$ with Fortune's Algorithm - Codeforces. (n.d.). Codeforces. <https://codeforces.com/blog/entry/85638>

Bose, P., Jansens, D., Van Renssen, A., Saumell, M., & Verdonshot, S. (2014). Making triangulations 4-connected using flips. *Computational Geometry*, 47(2), 187–197.
<https://doi.org/10.1016/j.comgeo.2012.10.012>

Aurenhammer, F., & Klein, R. (2000). Voronoi Diagrams**Partially supported by the Deutsche Forschungsgemeinschaft, grant K1 655 2-2. In Elsevier eBooks (pp. 201–290).
<https://doi.org/10.1016/b978-044482537-7/50006-1>

Boissonnat, J., Devillers, O., Dutta, K., & Glisse, M. (2020). Randomized incremental construction of delaunay triangulations of nice point sets. *Discrete and Computational Geometry*, 66(1), 236–268. <https://doi.org/10.1007/s00454-020-00235-7>

Example: Voronoi Diagram from Areas. (n.d.).
https://manifold.net/doc/mfd9/example_voronoi_diagram_from_areas.htm